SL2DT-06: Dashboard Conversion

> **Series:** SL2DT — Sumo Logic to Dynatrace | **Notebook:** 6 of 10 | **Created:** April 2026 | **Last Updated:** 06/23/2026

Overview

Goal of this step: rebuild in-scope Sumo dashboards in Dynatrace using Notebooks (for analytical dashboards) or Dashboards (for operational/executive views). Map each Sumo panel to a DQL section + visualization. Do **not** migrate every dashboard — only those that survived cut-scope (SL2DT-02).

The translation work leans on SL2DT-04 for the queries and on train-the-trainer for app-team delivery.

Table of Contents

Prerequisites

Requirement	Details
Audience	Core migration engineers + app-team dashboard authors
Inputs	`inventory/dashboards/` + translations from SL2DT-04 + cut-scope from SL2DT-02
Dynatrace access	Platform Token with `document:documents:write` (for notebooks/dashboards)
Prior reading	DASH-01 (Dashboard strategy), dynatrace-notebook-authoring skill

1. What You'll Produce

Artifact	Purpose
`dashboards/notebook-configs/`	Dynatrace Notebook JSON per migrated asset
`dashboards/dashboard-configs/`	Dynatrace Dashboard JSON (for op views)
`dashboards/variable-map.md`	Sumo dashboard variable → DT parameter mapping
`dashboards/rebuild-tracker.csv`	Per-dashboard status: owner, target, done/in-progress/blocked
`dashboard-rebuild-report.md`	Weekly progress to PM

2. Notebooks vs Dashboards — Choosing the Target

Dynatrace has two dashboard-like products. Pick deliberately.

Feature	Notebook	Dashboard
Purpose	Analytical, runbook, authoring	Operational, at-a-glance view
Navigation	Top-to-bottom reading	Panels in grid

Feature	Notebook	Dashboard
DQL editing	Inline	Inline
Markdown sections	Yes, freely interleaved	Text tiles only
Parameters/variables	Yes	Yes
Sharing	Document API	Document API
Suited for	Investigation, tutorials, reports	Monitoring displays, SRE pages

Decision Rule

Sumo Dashboard Type	Dynatrace Target
Investigation / analytical	**Notebook**
On-call operator view	**Dashboard**
Team home page	**Dashboard** (with embedded notebook links)
Executive KPI view	**Dashboard**
Post-incident forensics	**Notebook**
SRE runbook with manual steps	**Notebook**

3. Panel-to-Section Mapping

Each Sumo panel becomes one DQL section (with `visualization` field) in the notebook/dashboard.

Sumo Panel Anatomy

```
{
  "id": "panel-123",
  "title": "Error Rate by Service",
  "query": "_sourceCategory=prod/api | timeslice 1m | count by service",
  "visualization": "line",
  "position": { "x": 0, "y": 0, "width": 6, "height": 4 }
}
```

Dynatrace Notebook Section (Version 7)

```
{
  "id": "section-1",
  "type": "dql",
  "title": "Error Rate by Service",
  "state": {
    "input": {
      "value": "fetch logs, from:-1h | filter dt.source_entity ==
\"prod/api\" | makeTimeseries c = count(), interval:1m, by:{service.name}",
      "timeframe": { "from": "now()-1h", "to": "now()" }
    },
    "visualizationSettings": {
      "chartSettings": { "gapPolicy": "connect" }
    }
  },
  "visualization": "lineChart"
}
```

Mapping Workflow

For each panel:

1. Look up the Sumo query in `translations/all-queries.csv` → get the DQL
2. Add `from:` to the DQL (time range from Sumo panel config)
3. Map Sumo visualization type → Dynatrace visualization (see table below)
4. Preserve title, layout position, description
5. Skip panels marked `retire` in cut-scope

Common Mistakes

- **Copying the time range wrong.** Sumo dashboards have a global time picker that overrides panel-level ranges. Check both.
- **Missing `from:` clause.** The #1 translation error.
- **Stale DQL from SL2DT-04.** If you re-translated after schema changes, re-pull from the CSV.

4. Visualization Type Translation

Sumo Viz	Dynatrace Viz	Notes
Line chart	`lineChart`	Most common — direct
Area chart	`lineChart` with stacked fill	Same DQL, different chart setting
Bar chart	`barChart`	
Column chart	`barChart` (vertical)	
Pie chart	`pieChart`	Note: Dynatrace prefers donut/pie for proportions; tables for many categories
Single value	`singleValue`	With threshold colors
Table	`table`	
Honeycomb	`honeycomb`	Status grids
Heat map	`honeycomb` (approximation)	No direct equivalent for 2D heatmaps
Map (geo)	—	Not natively supported; use table with coordinates
Gauge	`singleValue` with min/max thresholds	
Text panel	Markdown section	

Example — Single Value with Threshold

Sumo: percentage panel showing `error_pct` with red if >5%, yellow if >1%, green otherwise.

Dynatrace Notebook:

```
// 4. Visualization Type Translation
fetch logs, from:-5m
| filter dt.source_entity == "prod/api"
| summarize total = count(), errors = countIf(contains(content, "ERROR"))
| fieldsAdd error_pct = 100.0 * toDouble(errors) / toDouble(total)
| fields error_pct
```

```
{
  "visualization": "singleValue",
  "visualizationSettings": {
    "thresholds": [
      { "value": 1, "color": "#FFB800" },
      { "value": 5, "color": "#DE350B" }
    ]
  }
}
```

5. Dashboard Variables → Parameters

Sumo dashboards use variables (e.g., `{{environment}}`, `{{host}}`). Dynatrace equivalents:

- **Notebook** — parameters (declared at notebook top, injected via `$param_name` in queries)
- **Dashboard** — variables (declared in dashboard config, injected via `$var_name`)

Mapping Example

Sumo variable:

```
{{environment}} → dropdown with values [prod, preprod, dev]
```

Dynatrace parameter:

```
{
  "parameters": [
    {
      "name": "environment",
      "type": "dropdown",
      "values": ["prod", "preprod", "dev"],
      "default": "prod"
    }
  ]
}
```

Usage in DQL:

```
// 5. Dashboard Variables → Parameters
fetch logs, from:-1h
| filter startsWith(dt.source_entity, concat($environment, "/"))
| summarize c = count(), by:{service.name}
```

Dependent Variables

Sumo supports variable dependencies (`{{host}}` filtered by `{{environment}}`). Dynatrace supports this via parameter chaining — declare the dependent parameter's query to reference the parent.

Multi-Value Variables

Sumo `{{host}}` with multi-select becomes DQL `in(f, $host)` where `$host` is a list-type parameter.

6. Rebuilding the Core Team's 5 Demo Dashboards

The train-the-trainer model (SL2DT-01 §4) requires the core team to rebuild 5 real-world dashboards as worked examples. Choose:

1. **A simple operational dashboard** — 4–6 panels, all line charts, one service
2. **A multi-team dashboard** — requires variable for team selection
3. **A complex analytical dashboard** — 10+ panels, mixed viz types
4. **An SLA dashboard** — singleValue + thresholds + burn rate

5. **A capacity dashboard** — metric queries, host-level detail

For each, produce:

- The raw Sumo → DT translation (what the translator did)
- The refined DT version (what an engineer polished)
- A README with lessons learned

These become the app-team reference library.

Demo Dashboard Template Structure

```
demos/
├── 01-simple-operational/
│   ├── sumo-original.json       # Exported from Sumo
│   ├── dt-raw-translation.json  # Post-translation, pre-polish
│   ├── dt-final.json            # What shipped
│   └── README.md                # Walkthrough
├── 02-multi-team/
├── 03-complex-analytical/
├── 04-sla/
└── 05-capacity/
```

7. App-Team Playbook (Train-the-Trainer)

Give each app team this playbook:

Dashboard Conversion — 7 Steps

1. **Identify your dashboards** in `cut-scope.md` (filter by your team)
2. **Locate translations** in `translations/all-queries.csv` (filter by your dashboard IDs)
3. **Pick target** — Notebook (analytical) or Dashboard (operational)
4. **Start from a demo template** — pick the closest of the 5 core-team demos
5. **Rebuild panel-by-panel** — for each panel, paste the DQL from translations, set visualization, title, position
6. **Add variables/parameters** if the original had them
7. **Submit PR to core team** for review — include screenshot + link + owner signoff

PR Checklist for App Teams

- [ ] All in-scope panels recreated
- [ ] Each panel's DQL validated (runs successfully against tenant)
- [ ] Variables mapped to parameters
- [ ] Title, description, metadata preserved
- [ ] Screenshot attached showing working state
- [ ] Link to live notebook/dashboard in tenant

Common App-Team Mistakes

- Using `parse` at query time when a FER (SL2DT-03) already extracted the field
- Forgetting `from:` on fetch/timeseries (Sumo has no equivalent; it's the #1 cause of query failures)
- Copying Sumo panel time ranges that made sense in Sumo but not in Dynatrace (e.g., Sumo's "all time" → DT needs a concrete `from:`)
- Skipping the cut-scope review ("I want to rebuild everything") — slows them down and burns budget

8. Step Exit Criteria

G6 — Dashboards Rebuilt

- [ ] Core team's 5 demo dashboards complete and committed to `demos/`
- [ ] All Wave 3 (top-usage) dashboards rebuilt and validated
- [ ] App-team Wave 4 dashboards: ≥75% complete, others tracked in `rebuild-tracker.csv`
- [ ] Variable/parameter mapping documented for every multi-parameter dashboard
- [ ] Every rebuilt dashboard: owner signoff recorded
- [ ] Aggregate metric: dashboard-conversion fidelity ≥90%

Next step: SL2DT-07 — User Governance & Access (IAM groups, bucket-scoped policies, SSO).

---

11. References

Dynatrace visualization surfaces

- Dashboards and notebooks (DT docs)
- Notebooks (DT docs)
- DQL Reference (DT docs)

Sumo Logic dashboard reference (source)

- Sumo Logic dashboards (Sumo Logic docs)

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace or Sumo Logic. Always verify information against the official Dynatrace documentation and Sumo Logic documentation.